

Nitro Python Runtime Plugin

Technical Documentation

Questions or issues? Get help quickly on [Discord](#).

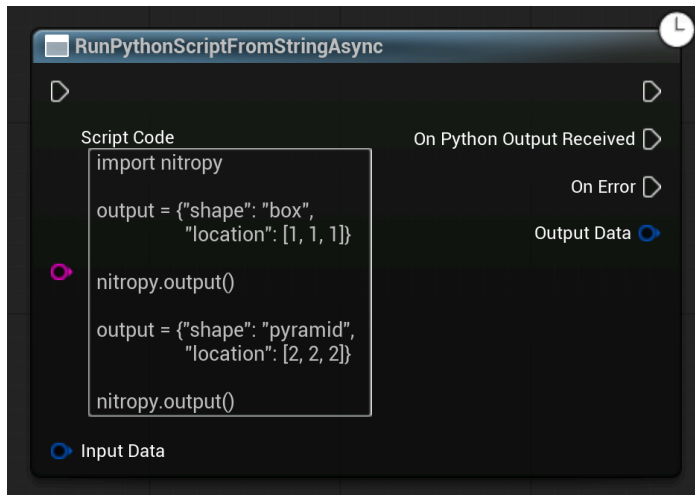
Index

List of Blueprint Nodes	1
Async Blueprint Nodes	2
Synchronous Blueprint Nodes	3
Input Variables to Python	4
Output Variables from Python	5
Output Error Info	6
Data Types	6
Plugin Directory Location	7
Customize Python Installation	7
Package Your Project	8
Logging	9

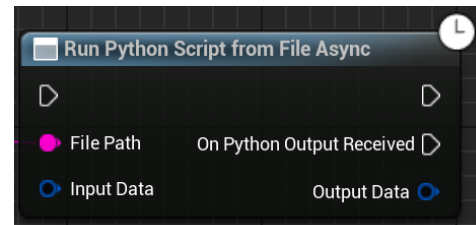
List of Blueprint Nodes

- Run Python Script From String Async
 - Run Python Script From File Async
 - Run Python Script From String
 - Run Python Script From File
-

Async Blueprint Nodes



Blueprint node: Run Python Script from String Async



Blueprint node: Run Python Script from File Async

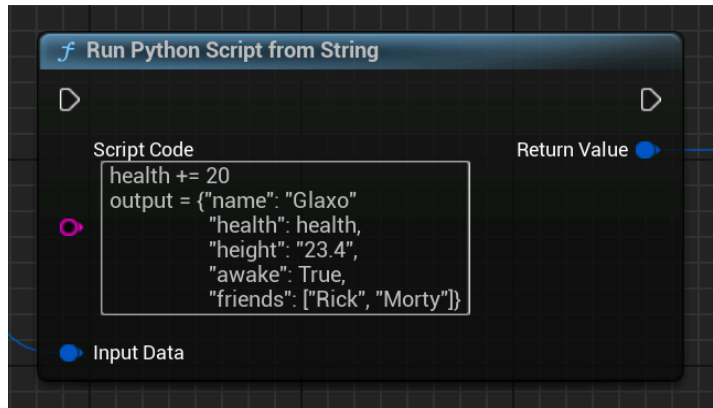
The **Output Data**  pin represents the **output** Python variable at each point that **nitropy.output()** is executed within the Python script.

The **On Python Output Received**  pin fires every time **nitropy.output()** is executed within the Python script.

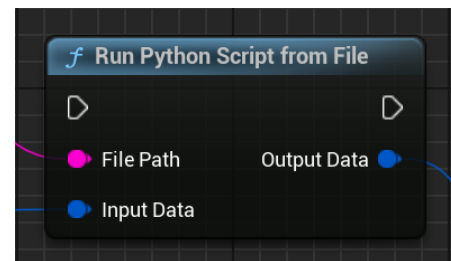
The **On Error**  pin fires when the Python script encounters an error. For info on how to extract error information from the output, see the section titled **Output Error Info**.

The async blueprint nodes run on a background thread, ensuring they don't introduce latency into the game. When your use case allows, it's recommended to use these async nodes instead of their synchronous counterparts to maintain optimal performance.

Synchronous Blueprint Nodes



Blueprint node: Run Python Script from String



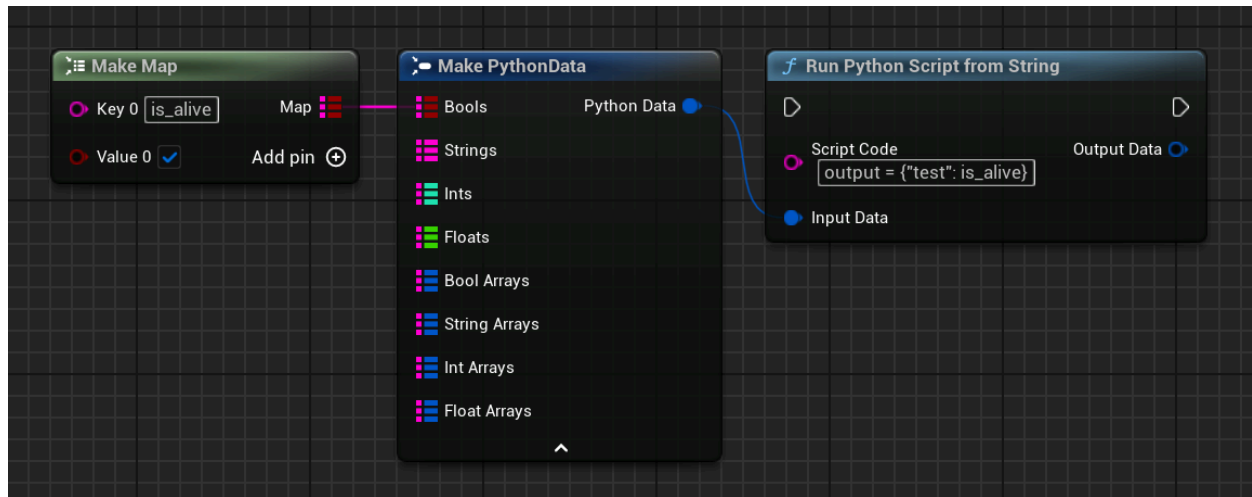
Blueprint node: Run Python Script from File

The **Output Data** pin represents the `output` variable from the Python script when the script finishes executing.

The synchronous blueprint nodes execute on the game thread, so they could cause latency in the game if the Python script takes a while to complete.

To output error info, see the section titled **Output Error Info**.

Input Variables to Python

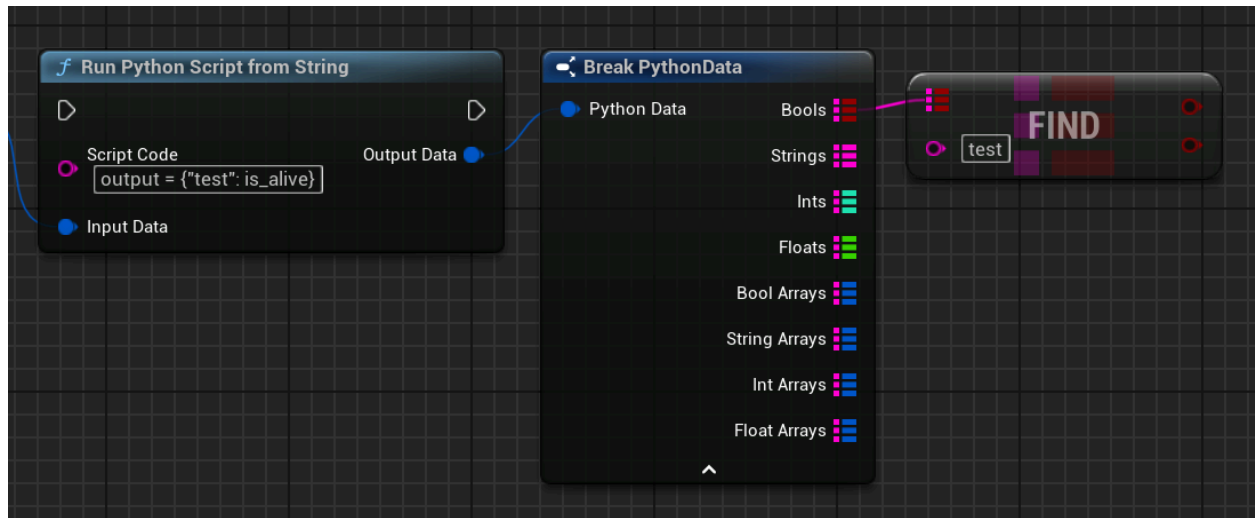


Example: send data from Unreal Engine to Python

To build an input to Python, drag from an **Input Data** pin, and add a **Make PythonData** node. Next, drag from the variable data type you would like to input, and add a **Make Map** node. The key represents the variable name in Python, and you can set the value in the mapping. This supports any number of variables.

In the example above, an Unreal Engine boolean value is passed to a Python variable named `is_alive`. The `is_alive` variable can then be used within the Python script. The script sets the `output` variable to a dictionary containing a key named "test", where the value is set to the `is_alive` variable which came from Unreal Engine.

Output Variables from Python

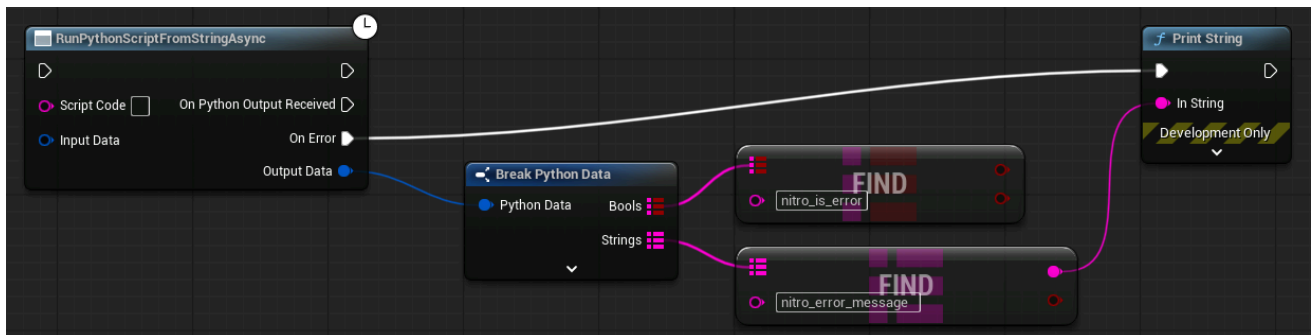


Example: send data from Python to Unreal Engine

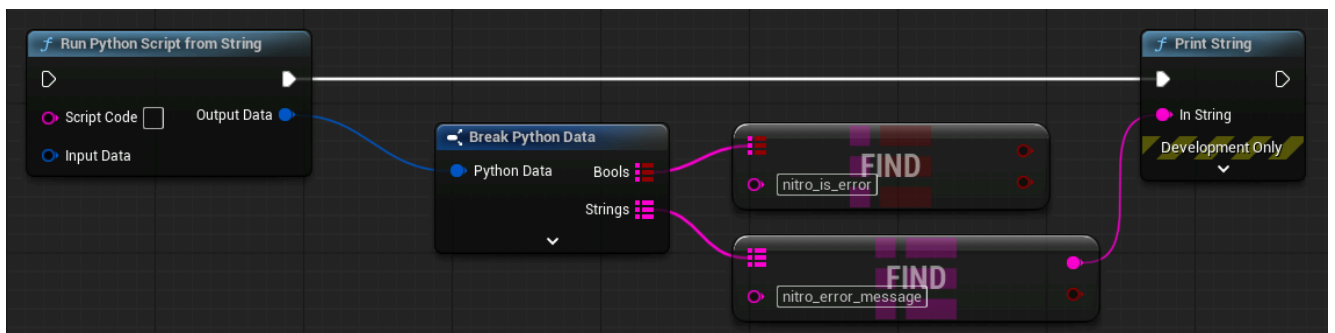
To get output from Python, drag from an **Output Data** pin, and add a **Break PythonData** node. Next, drag from the variable data type you wish to extract, and add a **Find** node. Enter the name of the key from the **output** variable in Python. The **Find** node then provides the value that was outputted from Python.

Output Error Info

The blueprint nodes always output error information, available by reading the keys **nitro_is_error** (bool indicating if error is present) and **nitro_error_message** (string containing traceback and error text). See the examples below for each blueprint node type.



Example: output error info from blueprint node (async)



Example: output error info from blueprint node (synchronous)

Data Types

The supported data types which can be passed between Unreal Engine and Python are bools, strings, ints, floats, and arrays of these types.

Plugin Directory Location

When installed from Fab, the plugin is located at:

`<Path_to_UnrealEngine>/Engine/Plugins/Marketplace/NitroPyt<hash>/`

The plugin uses Unreal Engine's built-in Python installation, located at:

`<Path_to_UnrealEngine>/Engine/Binaries/ThirdParty/Python3/Win64/`

Customize Python Installation

To add python packages (e.g., numpy, pandas, etc.) for use with the plugin, you can:

- **Follow this guide:**

 [Python Install Modules With PIP - Unreal Engine 5 Tutorial](#)

- **Or run this in Command Prompt (not Powershell):**

```
"<Path_to_UnrealEngine>\Engine\Binaries\ThirdParty\Python3\Win64\  
python.exe" -m pip install MODULENAME
```

Unreal Engine 5.4+ uses Python **3.11.8**; Unreal Engine 5.1-5.3 uses Python **3.9.7**.

Package Your Project

The plugin automatically embeds Python into the packaged build. You can package your project using the **Package Project** option from the Unreal Editor menu as usual.

Required Step for End Users: Add Python DLL Folder to User PATH

After packaging, anyone who runs the game must add a folder to their User PATH environment variable. This step is required for the plugin to work correctly and to avoid errors like **python3x.dll was not found** when opening the packaged .exe.

1. Open Environment Variables

- Press Win and search for **Edit the system environment variables**
- In the System Properties window, click **Environment Variables...** at the bottom

2. Edit User PATH

- In the top section labeled **User variables**, select Path and click **Edit...**

3. Add a New Path

- Click **New** and paste the following path, adjusting it to your actual installed game location:

`<Path_to_PackagedGame>\Windows\Engine\Binaries\ThirdParty\Python3\Win64`

4. Apply Changes

- Click **OK** on all windows to save

5. Launch the Game

- Double-click the .exe file to start the game

Logging

By default, the plugin logs everything, including Python scripts and print statements, to Unreal Engine's Output Log each time a Python script is executed. If you're running scripts continuously in a loop, you may want to reduce the amount of logging. There are 2 settings available:

Log Level:

- **None:** No logging
- **Error:** Only errors are logged
- **Warning:** Warnings and errors are logged
- **Verbose:** Detailed logging including Python scripts and print statements

Max Log Output Length:

- Maximum characters to display in logs (helps prevent spam)

These settings can be found in **Edit > Project Settings > Plugins > Nitro Python Runtime**, or search for "**Log Level**" or "**Max Log Output Length**" in Project Settings.

Questions or issues? Get help quickly on [Discord](#).